The Limits of AI-Assisted Development: Why Manual Testing Remains Necessary, Even With Frontier Models

Author: Jerry Abramson (with Claude Code)

Date: 2026-07-20

Context: Boston University research on local LLM tooling for agentic workflows, `iperf3KMP` project. Companion document to `LMSTUDIO_MCP_CASE_STUDY.md` and `PRINTF_FORMATTING_CASE_STUDY.md` — both are cited here as evidence, not re-derived. Where those two documents each argue a point about *one* comparison (local-model reasoning capability in the first; a specific interop-boundary defect class in the second), this document names a claim that cuts across both and does not go away as model quality improves: **there is a category of correctness that no AI assistant — local or frontier, weak or strong — can verify on its own, because verifying it requires a human to look at a running system and perceive it.**

1. Overview

The two companion case studies in this repository are sometimes read as being about model *capability* — a weaker local model failed a debugging task (`LMSTUDIO_MCP_CASE_STUDY.md` §11), a weaker local model shipped a silently broken printf call (`PRINTF_FORMATTING_CASE_STUDY.md` §5–§6). That reading is correct as far as it goes, but it invites a comfortable, incomplete conclusion: "a stronger model would have caught it." This document argues that conclusion is only half true, using a sharper, more recent, and more inconvenient data point than either prior case study offers — one produced by Claude, the frontier model being used to write this very sentence, in this same repository, a few hours before this document was started.

2. The Data Point: A Frontier Model's Own Unverified Fix

`PRINTF_FORMATTING_CASE_STUDY.md` §13 documents a second bug in this codebase, unrelated to the printf/vararg defect: the "iperf3 Output" panel's fixed-width columns rendered correctly on iOS and desktop, but misaligned on Android. The root cause — `mesloFontFamily()` falling back to the generic, per-backend `FontFamily.Monospace` alias instead of the bundled `meslonerdfont.ttf` already sitting unused in the resource tree — was diagnosed and fixed in this session, by Claude, using real git evidence rather than guesswork (§13 traces the dead, unused resource import all the way back

to the first KMP-port commit).

That fix is almost certainly correct. It is also, as of this writing, **verified in exactly one way: it compiles cleanly on all three targets.** It has not been run on an Android emulator or device. Nobody — human or model — has looked at the rendered "iperf3 Output" panel since the change was made. §13 says this explicitly, because the printf case study's own stated standard (§8: "every claim above about what works was checked against real execution, not just compilation, before being treated as done") requires saying it explicitly.

This is the load-bearing example for this entire document, for one reason: **the model that produced this unverified fix is not a 35B local model running on a devkit.** It is the same `claude-sonnet-5` frontier model discussed favorably throughout `LMSTUDIO_MCP_CASE_STUDY.md` §11 as the one that *could* do the sustained, multi-hop diagnostic reasoning the local models could not. Model strength closed the reasoning gap in that case study. It did not, and structurally cannot, close this one. The gap here is not "the model didn't reason carefully enough about whether the fix was right" — the reasoning (§13's git archaeology, the Skia-vs-Minikin rendering explanation) is sound. The gap is that reasoning about code is not the same activity as looking at a screen, and no amount of the former substitutes for the latter.

3. A Taxonomy of What Can Be Verified Without a Human

The two prior case studies, plus §2 above, span four distinct verification categories, worth separating because they respond differently to "just use a better model":

1. **Statically checkable (compiler/type system).** Argument types, syntax, structural well-formedness. AI assistants are excellent here, often faster and more thorough than a human at this specific class — but note this was never where either bug in this repo lived. The printf vararg bug (`PRINTF_FORMATTING_CASE_STUDY.md` §6) and the font-alias bug (§13) both *compiled cleanly*. This category catching nothing is precisely what made both bugs dangerous.
2. **Verifiable by automated test execution.** Logical/functional correctness for anything an assertion can express — the 270-line `StringFormatterTest.kt` suite, the 35/35 and 33/33 test runs cited in `PRINTF_FORMATTING_CASE_STUDY.md` §8. AI can write these tests, run them, and correctly interpret the results. This is real, valuable, and was done properly in this repo's own history.
3. **Not verifiable without direct human perceptual observation.** Anything whose correctness criterion is perceptual or experiential rather than propositional: does this column line up, is this

contrast readable, does this animation feel smooth, does this *look* like competent engineering. No assertion can encode "a human glancing at this would perceive it as correct" — the ground truth literally is a human's perception, not a computable predicate. §13's font bug lives entirely in this category. So, category-wise, does §5's gibberish-text bug, even though its *cause* was a logic error — the thing that first surfaced it in practice was a person looking at a phone screen (`docs/evidence/ios-printf-gibberish-output.png`), not a test failure.

4. **Sustained multi-hop reasoning under uncertainty.** Correlating asynchronous evidence, reading unfamiliar third-party source, revising a hypothesis after each individually-plausible fix turns out to be necessary-but-insufficient — `LMSTUDIO_MCP_CASE_STUDY.md` §4–§8 in full. This is the category where model *strength* mattered and where the cloud/local gap in that document's §11 was actually measured.

Category 4 is capability-dependent and, on the evidence in `LMSTUDIO_MCP_CASE_STUDY.md` §11.1–§11.2, is closing as models improve. Category 3 is not capability-dependent at all — it is a category error to ask a model to close it by getting smarter, because the missing ingredient isn't intelligence, it's a sensory channel: nothing in how an LLM is built gives it eyes on a running device. §2's example is the proof: this document's own author-model is, by the LMSTUDIO case study's own account, on the good side of the category-4 gap, and it still produced a category-3-unverified change.

4. Why "Just Show It a Screenshot" Only Partially Closes the Gap

A multimodal model can, in fact, look at a screenshot and correctly judge whether columns are aligned — this is exactly how `PRINTF_FORMATTING_CASE_STUDY.md` §5's true symptom was established, from the user's own screenshot, after the case study's original text had the failure mode wrong. So it would be inaccurate to say AI has *no* path into category 3.

What it lacks is the ability to independently decide *when* to go look, and the means to actually go look without a human closing the loop:

- It has no way to know a category-3 question exists at all unless someone thinks to ask "but does it actually look right" — the type checker and test suite give no signal that a perceptual question is even in play, which is exactly why §13's bug went unnoticed from the first KMP-port commit onward.
- It cannot itself build, install, launch, and navigate a phone app to the screen in question. Doing that requires either a human physically doing it, or a device-automation harness (emulator

control, screenshot capture, UI-tree inspection) that someone has to build and maintain — which is itself nontrivial engineering effort, and which this project, at its current stage, does not have.
- Even with such a harness, "does this look professionally finished" is a judgment a human makes about *impression*, not just about pixel positions — a screenshot-diffing tool can catch that column X moved five pixels; it cannot catch that the whole panel now reads as "sloppy" to someone deciding whether to keep funding the project (`PRINTF_FORMATTING_CASE_STUDY.md` §13's closing point).

5. Cost Asymmetry, and Where It's Heading

`PRINTF_FORMATTING_CASE_STUDY.md` §11 and `LMSTUDIO_MCP_CASE_STUDY.md` §10 both report real, measured session costs for categories 1, 2, and 4 work: $7.54 / ~20 minutes of API compute for the printf architecture-and-fix session, $5.41 / ~18 minutes for the MCP infrastructure-debugging session. Both figures are real `/cost` output, not estimates. Categories 1, 2, and 4 — compiling, testing, and even the kind of sustained diagnostic reasoning `LMSTUDIO_MCP_CASE_STUDY.md` documents — are, on this evidence, now cheap: single-digit dollars and minutes, for work that would otherwise cost a developer substantially more wall-clock time, and that cost keeps falling as models improve.

Category 3 has no analogous cost curve, because its cost isn't tokens — it's a human's actual attention, pointed at a running system. That cost does not fall as models get better, because better models were never the bottleneck for it. The practical implication for a project (or a team) leaning increasingly on AI-assisted development: as categories 1/2/4 keep getting cheaper and more automated, category 3 does not shrink to match — it becomes a *proportionally larger* share of total verification cost and risk, precisely because everything around it is being automated away while it is not. §13's bug is a small, low-stakes illustration of what that looks like in practice: a defect that survived clean compilation, a passing test suite, and a capable model's own correct-sounding diagnosis, and would still be sitting there today if nobody had looked at the actual screen.

6. Relevance to the Research

This document's claim is narrower than, and complementary to, `LMSTUDIO_MCP_CASE_STUDY.md`'s central question. That document asks whether locally-hosted models can match cloud frontier models on a given task class (they could not, on the evidence in its §11). This document asks a different question, orthogonal to model choice entirely: for a given piece of AI-assisted work, cloud or local,

how much of its correctness is even in principle verifiable by *any* model, versus how much structurally requires a human's own eyes — and whether "use a better model" is a meaningful answer to that second part (it is not). `PRINTF_FORMATTING_CASE_STUDY.md`'s two bugs, one from a local model (§5–§6) and one from Claude itself (§13, this document's §2), sit on opposite ends of the local/cloud capability spectrum this research is otherwise measuring, and land in the same unverified place — which is the point: model capability and human-verification necessity are separate axes, not substitutes for one another, and any research or engineering process that treats a stronger model as a replacement for a human looking at the actual, running product is measuring the wrong thing.

7. Cost of This Document's Own Session

Mirroring `PRINTF_FORMATTING_CASE_STUDY.md` §11 and `LMSTUDIO_MCP_CASE_STUDY.md` §10: real `/cost` output, covering the session that diagnosed and fixed §13's font bug and wrote both this document and the §13 addendum to the printf case study — not an estimate.

Metric	Value
Total cost	$3.48
Total duration (API/compute)	15m 15s
Total duration (wall clock)	36m 20s
Code/config changes	108 lines added, 6 lines removed

Usage by model:

Model	Input tokens	Output tokens	Cache read	Cache write	Cost
`claude-sonnet-5`	3.5k	62.2k	4.4M	232.5k	$3.48
`claude-haiku-4-5`	575	22	0	0	$0.000

This session is the cheapest of the three documented in this repository ($3.48, vs. $7.54 for the printf

architecture session and $5.41 for the MCP debugging session), despite covering a wider span of work — a compile-only fix, a substantial addendum to an existing case study, and this entire document. That is itself a small, concrete illustration of §5's point: the categories this session's cost actually reflects (reading code, writing prose, git archaeology, compiling) are the cheap and shrinking ones. The one thing this session's cost figure does *not* reflect, and cannot, is the category-3 verification §2 flags as still outstanding — actually looking at the Android screen. That check, whenever it happens, will not show up in a `/cost` table at all.